

Week 10

3. SQL JOINS

Dr. Andreas S. Maniatis
Assistant Professor

School of Computer Science
COMP.2800 – Software Development [2026W]

Wednesday, March 11th, 2026 | 14:30 – 15:00 | ER 1118



University of Windsor

● Agenda

- • Recap on RDBMS and SQL
- JOINS
- Wrap-up
- What's Next



Part I Recap





SQL



University of Windsor

● Introduction to SQL (review)

- SQL was introduced in the early 1970s, to supplement the Relational Database Model
- It has a number of subsets:
 - DDL
 - DML
 - DCL
 - TCL
- **DML**
 - SELECT
 - FROM
 - WHERE
 - ORDER BY



Part II

JOINS



JOINS: An Introduction



SQL

Database Querying

- SELECT
- DISTINCT
- WHERE
- LIKE
- ORDER BY
- LIMIT
- UNION

Database Manipulation

- CREATE TABLE
- INSERT INTO TABLE
- UPDATE
- DELETE
- DROP



● Multi-table Queries

- • How to access data that spans across multiple tables?



Working with Two Tables

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017



Working with Two Tables

```
SELECT *  
FROM students, takes  
WHERE student.ID = takes.ID;
```

- List the names of students along with the course ID of the courses that they took

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Working with Two Tables

```
SELECT *  
FROM   students, takes  
WHERE  student.ID = takes.ID;
```



ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology



Working with Two Tables

```
SELECT *  
FROM student, takes  
WHERE student.ID = takes.ID;
```

Same query with “natural join”

```
SELECT *  
FROM student natural join takes;
```



ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

○ Theory on Relational Algebra



Intuition from Relational Algebra

- The query plan for a SQL JOIN has at least the following Relational Algebra operators:
 - Projection (π): The subset of columns from tables. This corresponds to the column list after the SELECT keyword in the query.
 - Selection (σ): The subset of rows filtered by a condition. This comes from conditions defined in the WHERE clause in the query.
 - Join ($\bowtie$): the join between the tables coming from the JOIN operation.
- The key to optimizing a SQL JOIN query is to minimize the sizes of the tables fed to the JOIN ($\bowtie$) operator. Following best practices are designed to do precisely that.



Relational Algebra reminder: Projection (π)

- We can use Projections (π) to normalise a relation
- If $R(A,B,C)$ satisfies $A \rightarrow B$ then we can project it on A,B and A,C without losing information
- Lossless decomposition:

$$R = \pi_{AB}(R) \bowtie \pi_{AC}(R)$$

where $\pi_{AB}(R)$ is projection of R on AB and $\bowtie$ denotes a natural join.

R			$\pi_{AB}(R)$	
A	B	C	A	B

Relational Algebra reminder: Selection (σ)

R

A	B	C	D
1	x	c	c
2	y	d	e
3	z	a	a
4	u	b	c
5	w	c	d

$\sigma_{C=D}(R)$

A	B	C	D
1	x	c	c
3	z	a	a

● Relational Algebra reminder: Product ($R_1 \times R_2$)

○ • Cartesian Product

R1

A	B
1	x
2	y

R2

A	C
1	w
2	v
3	u

$R1 \times R2$

A	B	A	C
1	x	1	w
1	x	2	v
1	x	3	u
2	y	1	w
2	y	2	v
2	y	3	u

Relational Algebra reminder: Natural Join ($\bowtie$)

- $R1 \bowtie R2 = \pi_{R1.A, B, C} \sigma_{R1.A = R2.A} (R1 \times R2)$

R1

A	B
1	x
2	y

R2

A	C
1	w
2	v
3	u

R1 $\bowtie$ R2

A	B	C
1	x	w
2	y	v

● Relational Algebra reminder: Natural Join ($\bowtie$)

- $R1 \bowtie R2 = \pi_{R1.A, B, C} \sigma_{R1.A = R2.A} (R1 \times R2)$

R1

A	B
1	x
2	y

R2

A	C
1	w
2	v
3	u

R1 $\bowtie$ R2

A	B	C
1	x	w
2	y	v

○ Theory on JOINS



JOINS

- We use the relationship between tables to query information from more than one table
- A JOIN relates or associates two tables producing a single table
- There are several types of JOINS:
 - INNER JOIN
 - OUTER JOIN
 - LEFT OUTER JOIN
 - RIGHT OUTER JOIN



Characteristics of a JOIN

- Tables are joined, row by row and side by side, by satisfying whatever JOIN conditions are imposed
- Non-matching rows are included or excluded based on the *join type*
- A typical JOIN condition specifies a **foreign key** in one table and the associated **primary key** in another
- If a JOIN's connecting columns contain nulls the nulls will never join (remember a null is unknown – so an unknown cannot equal another unknown)



Joining two tables

- Get all the results from the two tables joined on the **ids**

id	prod_name	qty	supplier
1	Chicken	2	1
2	Turkey	14	1
3	Beef	22	2

id	supplierName	contactName	contactPhone	contactEmail	contactPosition
1	Farm Chicken Supplier	John Doe	555-6656	jdove@email.com	Buyer
2	Cow Farms	Mike Smith	666-9656	msmith@email.com	manager

```
SELECT *  
FROM product, supplier
```

- This result set is called a **Cartesian Product** – the number of rows retrieved will be the number of rows in the first table (product) multiplied by the number of rows in the second table (supplier)

Joining two tables

- Cartesian product of two tables

id	prod_name	qty	supplier	id	supplierName	contactName	contactPhone	contactEmail	contactPosition
1	Chicken	2	1	1	Farm Chicken Supplier	John Doe	555-6656	jdoe@email.com	Buyer
1	Chicken	2	1	2	Cow Farms	Mike Smith	666-9656	msmith@email.com	manager
2	Turkey	14	1	1	Farm Chicken Supplier	John Doe	555-6656	jdoe@email.com	Buyer
2	Turkey	14	1	2	Cow Farms	Mike Smith	666-9656	msmith@email.com	manager
3	Beef	22	2	1	Farm Chicken Supplier	John Doe	555-6656	jdoe@email.com	Buyer
3	Beef	22	2	2	Cow Farms	Mike Smith	666-9656	msmith@email.com	manager

Cartesian Product (CROSS JOIN in SQL)

Customers

CustomerId	Name
1	Shree
2	Kalpana
3	Basavaraj

Orders

OrderID	CustomerId	OrderDate
100	1	2014-01-29 23:56:57.700
200	4	2014-01-30 23:56:57.700
300	3	2014-01-31 23:56:57.700

Cross Join

Result

CustomerId	Name	OrderID	CustomerId	OrderDate
1	Shree	100	1	2014-01-30 23:48:32.850
2	Kalpana	100	1	2014-01-30 23:48:32.850
3	Basavaraj	100	1	2014-01-30 23:48:32.850
1	Shree	200	4	2014-01-31 23:48:32.853
2	Kalpana	200	4	2014-01-31 23:48:32.853
3	Basavaraj	200	4	2014-01-31 23:48:32.853
1	Shree	300	3	2014-02-31 23:48:32.853
2	Kalpana	300	3	2014-02-31 23:48:32.853
3	Basavaraj	300	3	2014-02-31 23:48:32.853

JOIN Operations



Moving from WHERE to JOIN

```
SELECT Customers.Company,  
Customers.[Business Phone], Orders.[Order  
ID], Orders.[Order Date]  
FROM Customers  
    INNER JOIN Orders ON Orders.[Customer ID]  
= Customers.ID  
    WHERE Orders.[Order Date] < #4/15/2006#
```

Sources: MS Access
<https://sqliteonline.com/> (Use Northwinds)



JOIN Operations

- **Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An
FROM r1 natural join r2 natural join ... natural join rn
WHERE P;
```

```
SELECT *
FROM student, takes
WHERE student.ID = takes.ID;
```

Same query with “natural join”

```
SELECT *
FROM student natural join takes;
```

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

JOIN Operations

- **Join operations** take two relations and return as a result another relation.

SELECT $A_1, A_2, \dots, A_n$
FROM r_1 natural
WHERE P;

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

join r_n

SELECT *
FROM student, takes
WHERE student.ID = takes.ID;

Same query with “natural join”
SELECT *
FROM student natural join takes;

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

JOIN Operations

- **Join operations** take two relations and return as a result another relation.

SELECT $A_1, A_2, \dots, A_n$
FROM r_1 natural
WHERE P;

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

join r_n

SELECT *
FROM student, takes
WHERE student.ID = takes.ID;

Same query with "natural join"
SELECT *
FROM student natural join takes;

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

JOIN Operations

- **Join operations** take two relations and return as a result another relation.

SELECT $A_1, A_2, \dots, A_n$
FROM r_1 natural
WHERE P;

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

join r_n

SELECT *
FROM student, takes
WHERE student.ID = takes.ID;

Same query with “natural join”
SELECT *
FROM student natural join takes;

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

JOINS

- JOIN (a.k.a. INNER JOIN)

- LEFT JOIN

- RIGHT JOIN

- FULL JOIN

- SELECT statements can be very inefficient for databases, so we use JOIN



Three types of JOIN

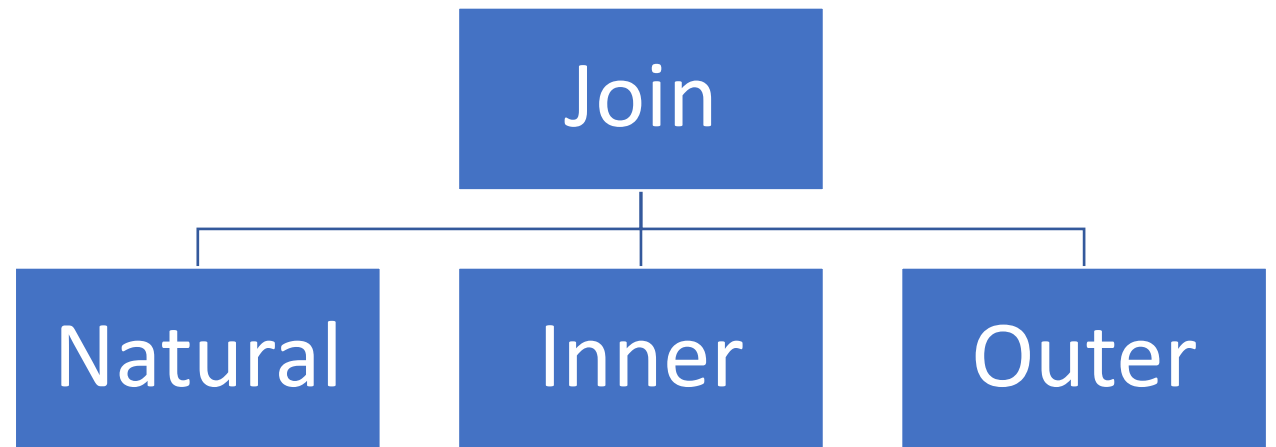
- **Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An
FROM r1 natural join r2 natural join ... natural join rn
WHERE P;
```

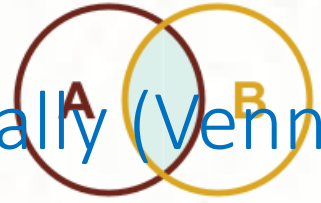
```
SELECT *
FROM student, takes
WHERE student.ID = takes.ID;
```

Same query with “natural join”

```
SELECT *
FROM student natural join takes;
```

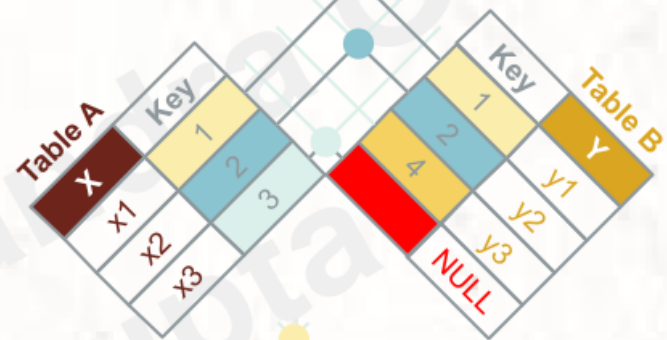
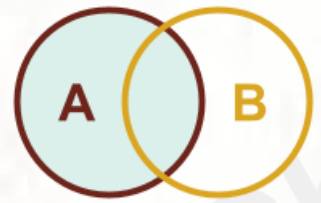


SELECT <columns>
FROM A
[INNER] JOIN B
ON A.Key = B.Key;



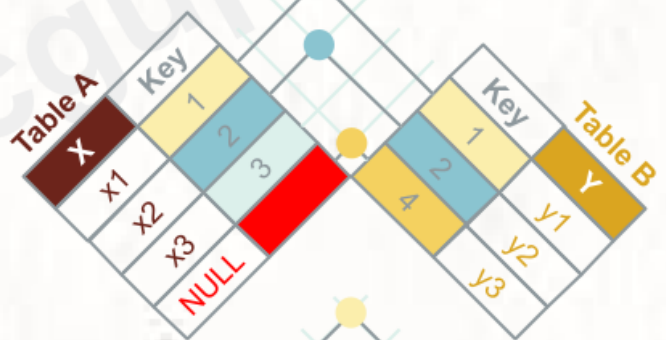
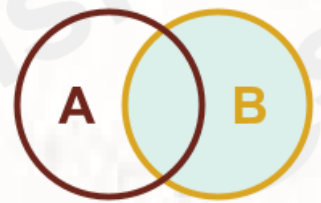
Key	X	Y
1	x1	y1
2	x2	y2

SELECT <columns>
FROM A
LEFT [OUTER] JOIN B
ON A.Key = B.Key;



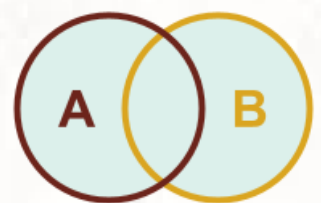
Key	X	Y
1	x1	y1
2	x2	y2
3	x3	NULL

SELECT <columns>
FROM A
RIGHT [OUTER] JOIN B
ON A.Key = B.Key;



Key	X	Y
1	x1	y1
2	x2	y2
4	NULL	y3

SELECT <columns>
FROM A
FULL [OUTER] JOIN B
ON A.Key = B.Key;



Key	X	Y
1	x1	y1
2	x2	y2
3	x3	NULL
4	NULL	y3

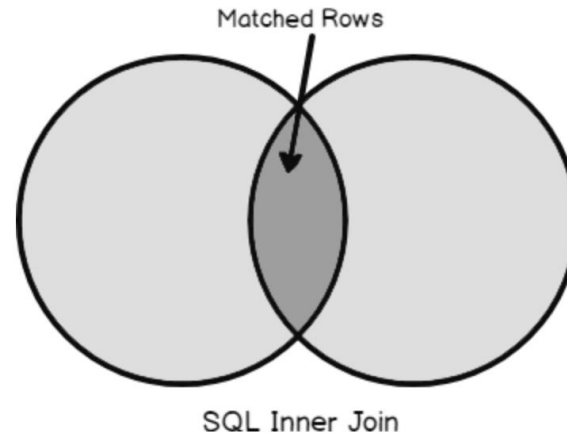


INNER JOIN



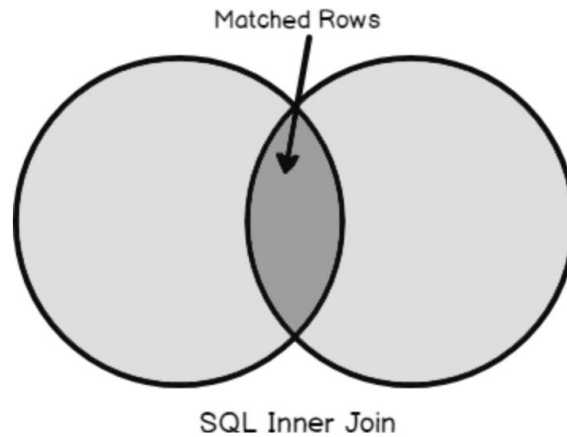
INNER JOIN

- **INNER JOIN** returns combined column values of two tables based on the *join condition* (predicate)



INNER JOIN

- **INNER JOIN** returns combined column values of two tables based on the *join condition* (predicate)



Query with “inner join”

```
SELECT *  
FROM student inner join takes  
on student.ID = takes.ID;
```

```
SELECT *  
FROM student, takes  
WHERE student.ID = takes.ID;
```

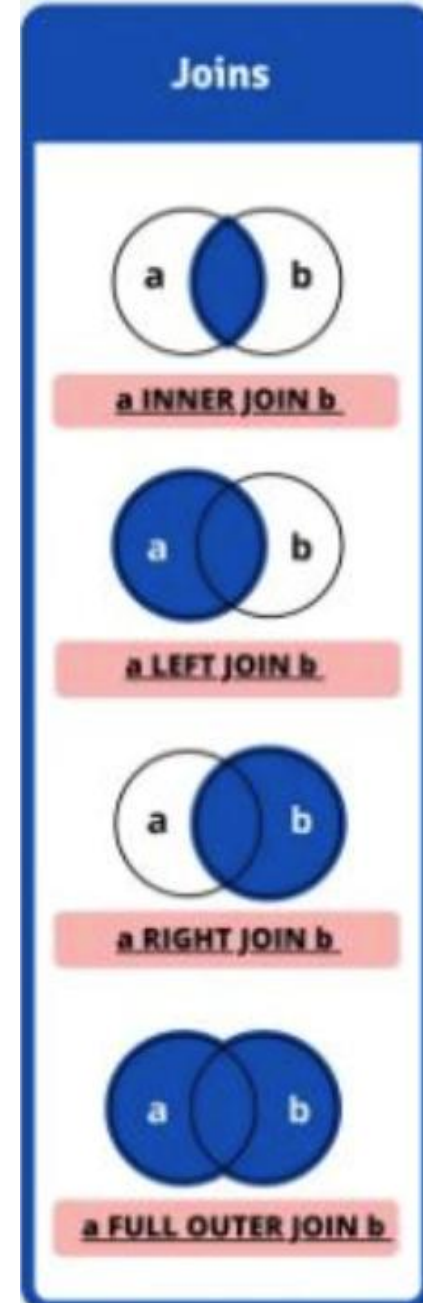
Same query with “natural join”

```
SELECT *  
FROM student natural join takes;
```

INNER JOIN

```
SELECT Column1, Column2, ...  
FROM Table1  
JOIN Table 2  
ON Table1.ColumnX = Table2.ColumnY;
```

- Must be at least **one match in BOTH** tables.



INNER JOIN

```
SELECT Employees.LastName, Orders.ShippedDate  
FROM Orders  
INNER JOIN Employees ON Orders.EmployeeID =  
Employees.EmployeeID
```

Reference:

<https://www.essentialsql.com/what-is-the-difference-between-a-right-and-left-outer-join/>



INNER JOIN

```
SELECT Employees.LastName,  
       Orders.ShippedDate  
FROM Orders  
      INNER JOIN Employees ON  
      Employees.EmployeeID
```

Orders and Employees INNER JOIN In Action

Orders		
OrderID	EmployeeID	ShippedDate
1	1	2012-01-05
2	2	2012-01-28
3	1	2012-02-23
4	4	2012-03-14
5	2	2012-04-06
6	3	2012-04-29
7	3	2012-05-22
8	4	2012-06-14
9	1	2012-07-07
10	2	2012-08-04
11	3	2012-08-22
12	4	2012-09-14
13	3	2012-10-07
14	2	2012-10-30
15	2	2012-11-22
16	4	2012-12-15
17	1	2013-01-07
18	3	2013-01-30
19	4	2013-03-01
20	1	2013-03-17

Employees	
EmployeeID	LastName
1	White
2	Lee
3	Smith
4	Baker

LastName	ShippedDate
White	2012-01-05
Lee	2012-01-28
White	2012-02-23
Baker	2012-03-14
Lee	2012-04-06
Smith	2012-04-29
Smith	2012-05-22
Baker	2012-06-14
White	2012-07-07
Lee	2012-08-04
Smith	2012-08-22
Baker	2012-09-14
Smith	2012-10-07
Lee	2012-10-30
Lee	2012-11-22
Baker	2012-12-15
White	2013-01-07
Smith	2013-01-30
Baker	2013-03-01
White	2013-03-17

Reference:

<https://www.essentialsql.com/what-is-the-difference-between-a-right-and-left-outer-join/>



● In-class Activity: INNER JOIN in Northwinds

- Display the company name, business phone number, order id, and order date for all companies that placed orders before Apr. 15, 2006.

Use the Customers and Orders tables.

Sources: MS Access
<https://sqliteonline.com/> (Use Northwinds)



● In-class Activity: INNER JOIN in Northwinds (Solution)

- Display the company name, business phone number, order id, and order date for all companies that placed orders before Apr. 15, 2006.

Use the Customers and Orders tables.

```
SELECT Customers.Company, Customers.[Business  
Phone], Orders.[Order ID], Orders.[Order Date]  
FROM Customers  
INNER JOIN Orders ON Orders.[Customer ID] =  
Customers.ID  
WHERE Orders.[Order Date] < #4/15/2006#
```

Sources: MS Access
<https://sqliteonline.com/> (Use Northwinds)





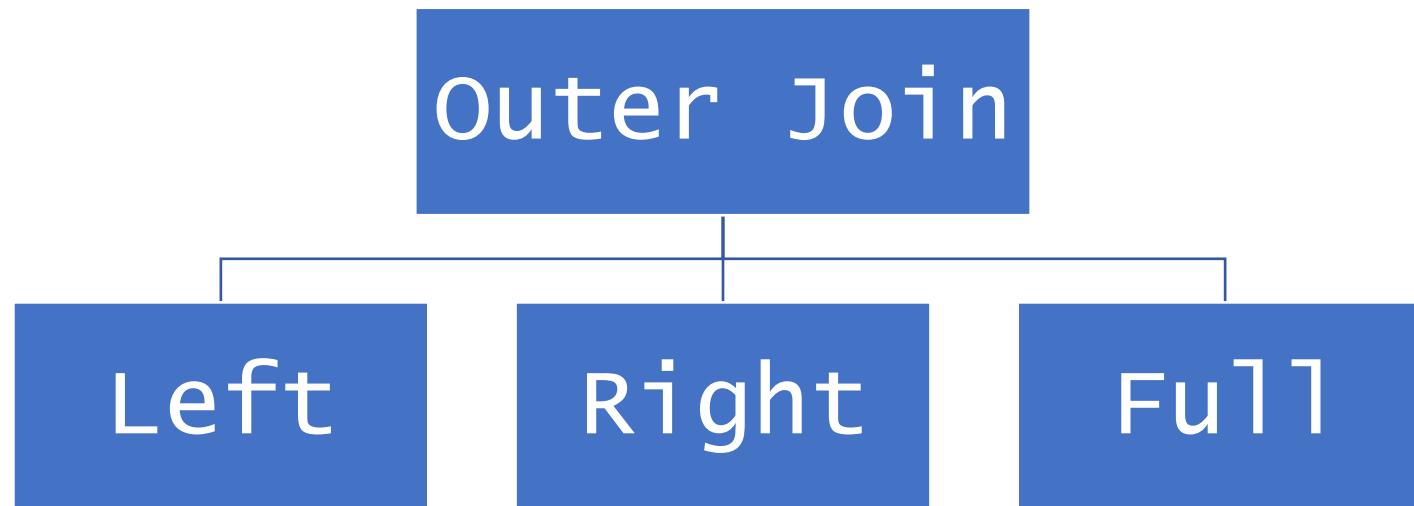
OUTER JOIN



University of Windsor

OUTER JOIN

- **OUTER JOINS** are joins that return matched values and unmatched values from either or both tables



Why do we need OUTER JOINS?

Query with "inner join"

```
SELECT *  
FROM student inner join takes  
on student.ID = takes.ID;
```

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

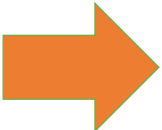


Why do we need OUTER JOINS?

Query with "inner join"

```
SELECT *  
FROM student inner join takes  
on student.ID = takes.ID;
```

Student Relation



ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017





LEFT JOIN



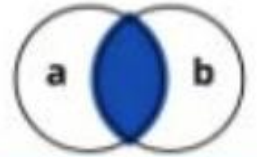
University of Windsor

LEFT JOIN

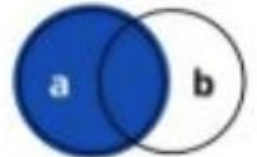
```
SELECT  Column1, Column2, ...  
FROM    Table1  
        LEFT OUTER JOIN  Table 2  
        ON      Table1.ColumnX =  
                Table2.ColumnY;
```

- ***All rows from left table and matching rows from right.*** Matching is not a requirement!
- a.k.a. LEFT OUTER JOIN

Joins



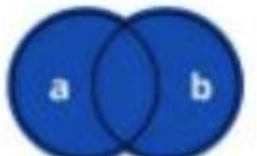
a INNER JOIN b



a LEFT JOIN b

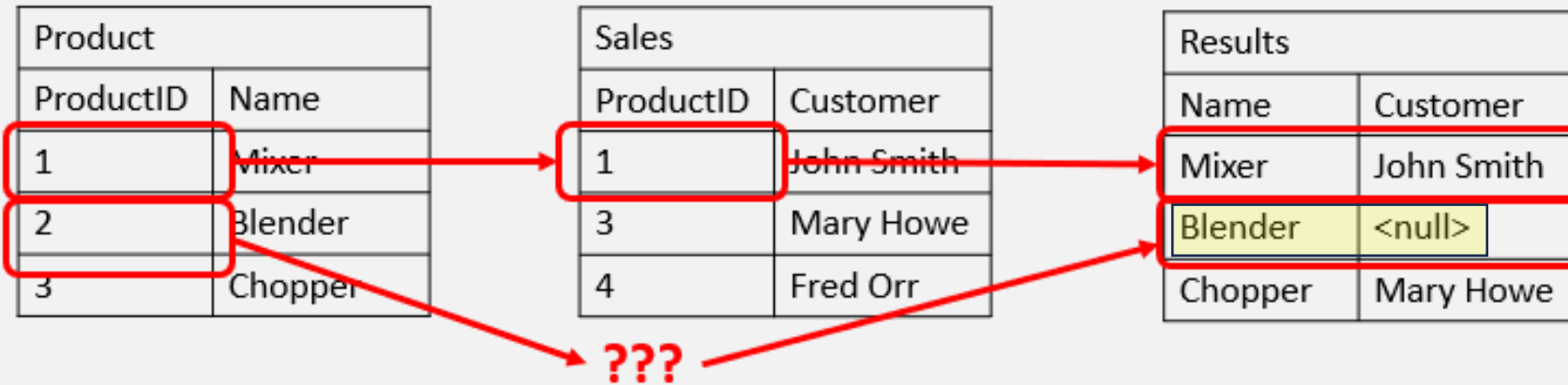


a RIGHT JOIN b



a FULL OUTER JOIN b

LEFT JOIN

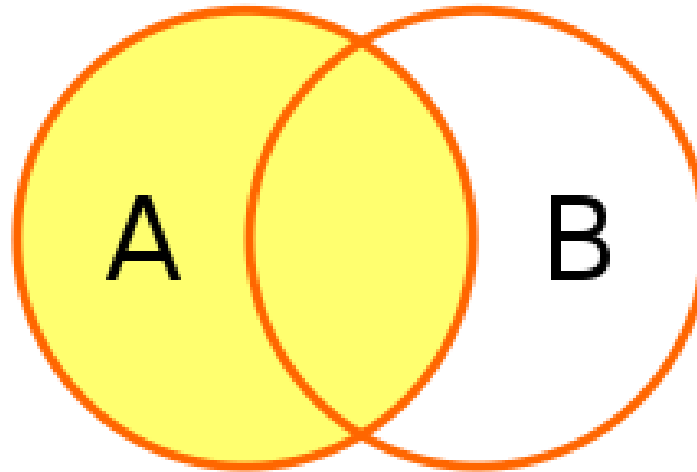


```
SELECT P.Name,  
       S.Customer  
FROM   Product P  
       LEFT OUTER JOIN Sales S  
       ON P.ProductID = S.ProductID
```

Reference:
<https://www.essentialsql.com/what-is-the-difference-between-a-right-and-left-outer-join/>

LEFT OUTER JOIN

- **LEFT OUTER JOINS** preserves unmatched rows from the first (left) table with NULL for missing row values of the second (right) table



LEFT OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017



LEFT OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3		
4		
5		

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017

Query with "outer join"

```
SELECT *  
FROM student left outer join takes  
on student.ID = takes.ID;
```

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
2	Shankar	ECE	NULL	NULL	NULL
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017

LEFT OUTER JOIN

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Find all students who have not taken a course

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
2	Shankar	ECE	NULL	NULL	NULL
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017



LEFT OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Find all students who have not taken a course

```
SELECT *  
FROM student left outer join takes  
on student.ID = takes.ID  
WHERE course_id is NULL;
```

ID	name	dept_name	course_id	semester	year
1	Tanaka	ECE			2017
1	Tanaka	ECE		Fall	2017
2	Shankar	ECE	NULL	NULL	NULL
3	Brandt	History	HIS-350	Fall	2017

ID	name	dept_name	course_id	semester	year
2	Shankar	ECE	NULL	NULL	NULL

● In-class Activity: LEFT JOIN in NORTHWINDS

- Display a list of every company name, and order ids that were placed by each company.
Sort by the Order ID field.
Use the Customers and Orders tables.

Be sure to show **EVERY** company.

Sources: MS Access
<https://sqliteonline.com/> (Use Northwinds)



In-class Activity: LEFT JOIN in NORTHWINDS (Solution)

- Display a list of every company name, and order ids that were placed by each company. Sort by the Order ID field. Use the Customers and Orders tables.

Be sure to show **EVERY** company.

```
SELECT Customers.Company, Orders.[Order ID]
FROM Customers
LEFT JOIN Orders ON Customers.ID =
                  Orders.[Customer ID]
ORDER BY [Order ID]
```

Sources: MS Access
<https://sqliteonline.com/> (Use Northwinds)





RIGHT JOIN



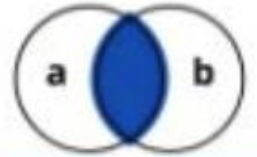
University of Windsor

RIGHT JOIN

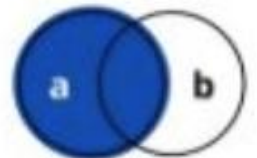
```
SELECT Column1, Column2, ...  
FROM Table1  
RIGHT JOIN Table 2  
ON Table1.ColumnX = Table2.ColumnY;
```

- ***All rows from right table and matching rows from left. Matching rows from left table is not a requirement***
- a.k.a. RIGHT OUTER JOIN

Joins



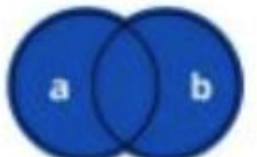
a INNER JOIN b



a LEFT JOIN b



a RIGHT JOIN b



a FULL OUTER JOIN b

RIGHT JOIN

Product	
ProductID	Name
1	Mixer
2	Blender
3	Chopper

Sales	
ProductID	Customer
1	John Smith
3	Mary Howe
4	Fred Orr

Results	
Name	Customer
Mixer	John Smith
Chopper	Mary Howe
<null>	Fred Orr

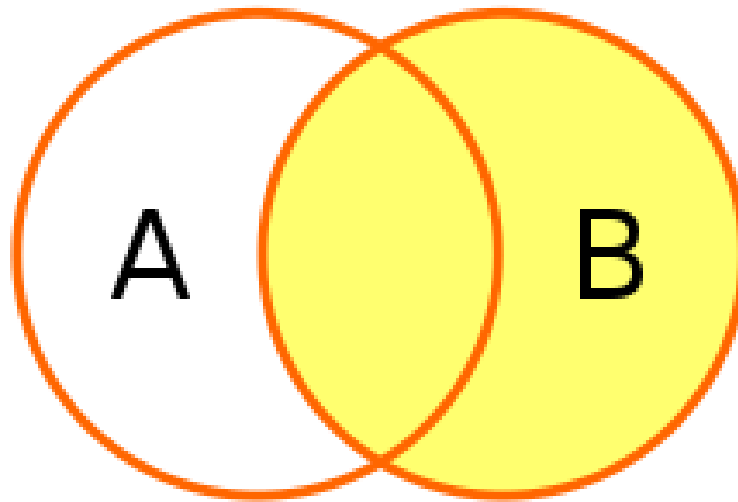
???

```
SELECT P.Name,  
       S.Customer  
FROM   Product P  
       RIGHT OUTER JOIN Sales S  
       ON P.ProductID = S.ProductID
```

Reference:
<https://www.essentialsql.com/what-is-the-difference-between-a-right-and-left-outer-join/>

RIGHT OUTER JOIN

- **Right OUTER JOIN** preserves unmatched rows from the second (right) table with NULL for missing row values of the first (left) table



RIGHT OUTER JOIN

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

RIGHT OUTER JOIN

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-199	Spring	2017
5			
5			

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History

Query with "right outer join"

```
SELECT *  
FROM takes right outer join student  
on takes.ID = student.ID;
```

RIGHT OUTER JOIN

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE

Query with "right outer join"

```
SELECT *  
FROM takes right outer join student  
on takes.ID = student.ID;
```

ID	course_id	semester	year	name	dept_name
1	CS-101	Fall	2017	Tanaka	ECE
1	CS-190	Fall	2017	Tanaka	ECE
2	NULL	NULL	NULL	Shankar	ECE
3	HIS-350	Fall	2017	Brandt	History
4	MU-120	Spring	2017	Jeff	Music
5	BIO-102	Fall	2017	Snow	Biology
5	BIO-191	Spring	2017	Snow	Biology





FULL JOIN

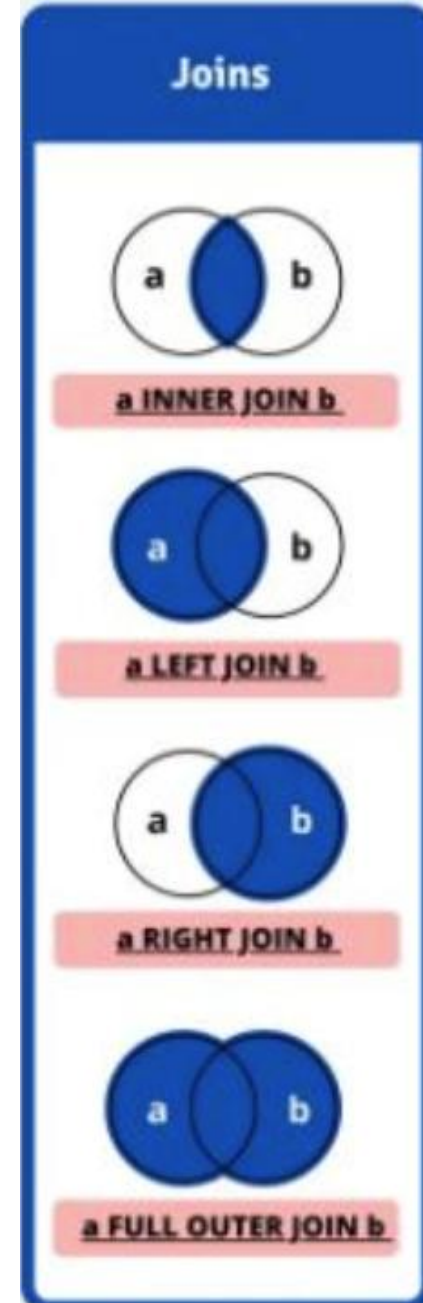


University of Windsor

FULL JOIN

```
SELECT Column1, Column2, ...  
FROM Table1  
FULL JOIN Table 2  
ON Table1.ColumnX = Table2.ColumnY;
```

- **Returns all rows from each table.**
Remember your math on matrices: Cartesian Product
- a.k.a FULL OUTER JOIN



FULL JOIN

```
select c.CompanyID, c.CompanyName, p.ProductID, p.ProductName
from @Company c
full outer join @Product p on c.CompanyID = p.CompanyID
order by c.CompanyName, p.ProductName
```

	CompanyID	CompanyName	ProductID	ProductName
1	NULL	NULL	8	Quick Books
2	NULL	NULL	9	Turbo Tax
3	2	Adobe	4	Acrobat
4	2	Adobe	5	Photo Shop
5	1	Microsoft	3	Access
6	1	Microsoft	1	Excel
7	1	Microsoft	2	SQL Server
8	3	Oracle	NULL	NULL
9	4	TechSmith	7	Camtasia
10	4	TechSmith	6	Snagit

"RIGHT JOIN"

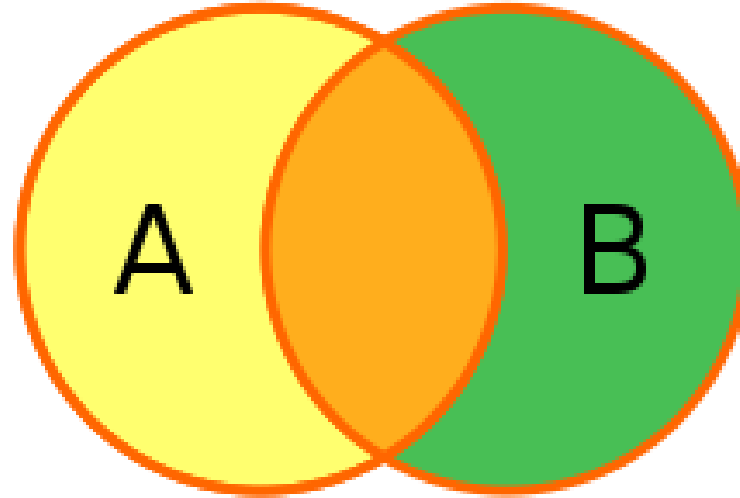
"INNER JOIN"

"LEFT JOIN"

Reference:
<https://www.essentialsql.com/sql-full-outer-join/>

FULL OUTER JOIN

- **Full OUTER JOIN** is a combination of left and right outer join



FULL OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017
6	HIS-120	Spring	2017

FULL OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History
4	Jeff	Music
5	Snow	Biology

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017
6	HIS-120	Spring	2017

Query with “full outer join”

```
SELECT *  
FROM student full outer join takes  
on takes.ID = student.ID;
```



FULL OUTER JOIN

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017
6	HIS-120	Spring	2017

Query with “full outer join”

```
SELECT *
FROM student full outer join takes
on takes.ID = student.ID;
```

			course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
2	Shankar	ECE	NULL	NULL	NULL
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017
6	NULL	NULL	HIS-120	Spring	2017

Part III

JOIN Visualizers

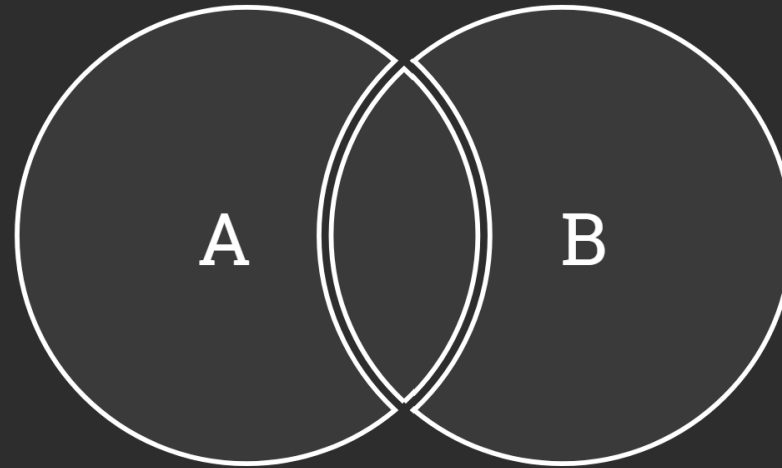


JOIN Visualizers



SQL Joins Visualizer

SQL Joins Visualizer help to you build SQL JOIN between two tables by using of Venn diagrams



Please select how do you want to do SQL JOIN between two table

Copy SQL

Reference: <https://sql-joins.leopard.in.ua/>

Visual JOIN

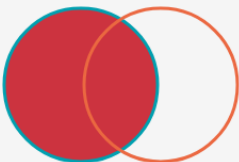
Understand how joins work by interacting and see it visually

INNER JOIN

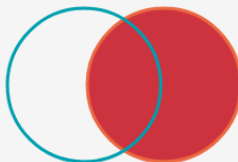
(or JOIN)



LEFT JOIN

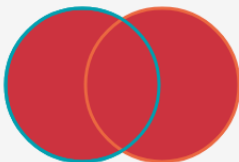


RIGHT JOIN



OUTER JOIN

(with UNION)



SQL

```
SELECT users.name, likes.like FROM users JOIN likes ON users.id = likes.user_id;
```

Description »

Users

ID	Name
1	Patrik
2	Albert
3	Maria
4	Darwin
5	Elizabeth

JOIN

Name	Like
Maria	Stars
Patrik	Climbing
Patrik	Code
Darwin	Apples

Likes

User ID	Like
3	Stars
1	Climbing
1	Code
6	Rugby
4	Apples

Part IV

Wrap-up – What's Next



Summary



Summary

In this lecture:

- We worked on the main Data Manipulation Language (DML) commands of SQL:
 - SELECT
 - FROM
 - WHERE
- We were introduced to SQL JOINS.
- Performed in-class exercises on SQL JOINS.



Three Types of Join

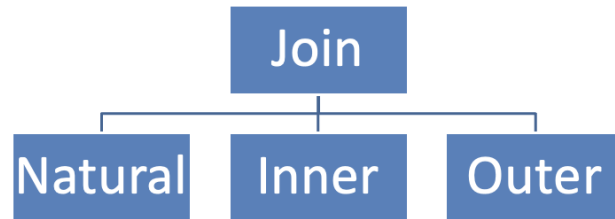
- **Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An  
FROM r1 natural join r2 natural join ... natural join rn  
WHERE P;
```

```
SELECT *  
FROM student, takes  
WHERE student.ID = takes.ID;
```

Same query with "natural join"

```
SELECT *  
FROM student natural join takes;
```



Three Types of Join

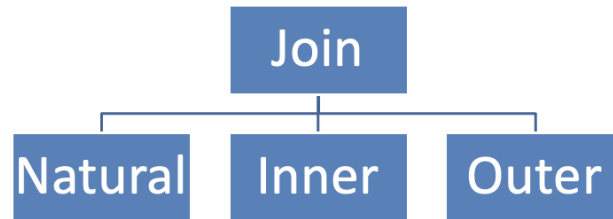
- **Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An
FROM r1 natural join r2 natural join ... natural join rn
WHERE P;
```

```
SELECT *
FROM student, takes
WHERE student.ID = takes.ID;
```

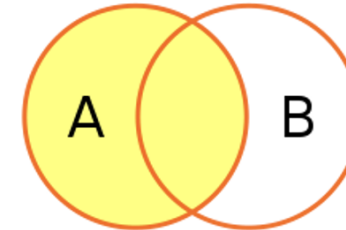
Same query with "natural join"

```
SELECT *
FROM student natural join takes;
```



Left Outer Join

- **Left OUTER JOIN** preserves unmatched rows from the first (left) table with NULL for missing row values of the second (right) table



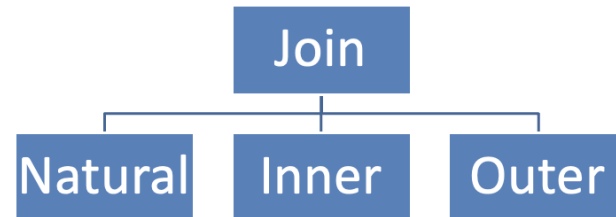
Three Types of Join

- **Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An  
FROM r1 natural join r2 natural join ... natural join rn  
WHERE P;
```

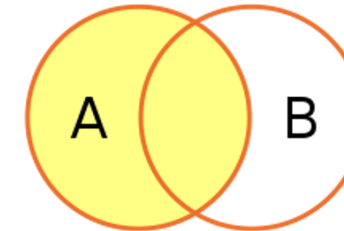
```
SELECT *  
FROM student, takes  
WHERE student.ID = takes.ID;
```

```
Same query with "natural join"  
SELECT *  
FROM student natural join takes;
```



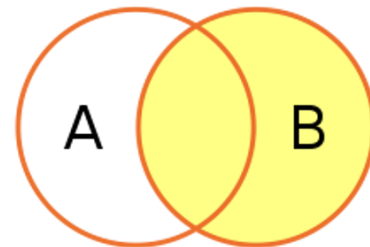
Left Outer Join

- **Left OUTER JOIN** preserves unmatched rows from the first (left) table with NULL for missing row values of the second (right) table



Right Outer Join

- **Right OUTER JOIN** preserves unmatched rows from the second (right) table with NULL for missing row values of the first (left) table



Three Types of Join

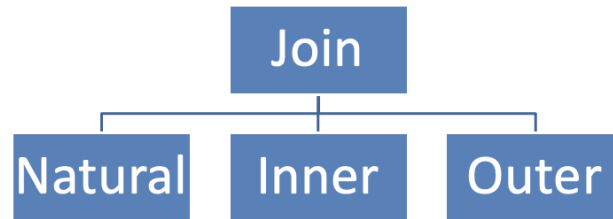
- Join operations** take two relations and return as a result another relation.

```
SELECT A1, A2, ... An
FROM r1 natural join r2 natural join ... natural join rn
WHERE P;
```

```
SELECT *
FROM student, takes
WHERE student.ID = takes.ID;
```

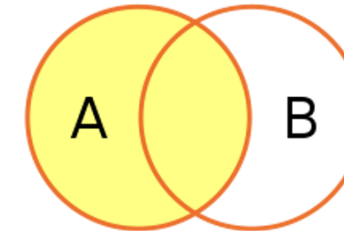
Same query with "natural join"

```
SELECT *
FROM student natural join takes;
```



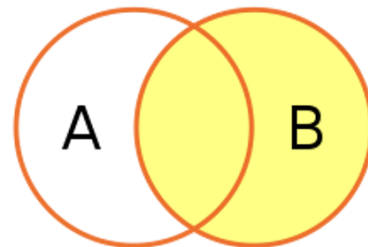
Left Outer Join

- Left OUTER JOIN** preserves unmatched rows from the first (left) table with NULL for missing row values of the second (right) table



Right Outer Join

- Right OUTER JOIN** preserves unmatched rows from the second (right) table with NULL for missing row values of the first (left) table



Full Outer Join

Student Relation

ID	name	dept_name
1	Tanaka	ECE
2	Shankar	ECE
3	Brandt	History

Takes Relation

ID	course_id	semester	year
1	CS-101	Fall	2017
1	CS-190	Fall	2017
3	HIS-350	Fall	2017
4	MU-120	Spring	2017
5	BIO-102	Fall	2017
5	BIO-191	Spring	2017
6	HIS-120	Spring	2017

Query with "full outer join"

```
SELECT *
FROM student full outer join takes
on takes.ID = student.ID;
```

			course_id	semester	year
1	Tanaka	ECE	CS-101	Fall	2017
1	Tanaka	ECE	CS-190	Fall	2017
2	Shankar	ECE	NULL	NULL	NULL
3	Brandt	History	HIS-350	Fall	2017
4	Jeff	Music	MU-120	Spring	2017
5	Snow	Biology	BIO-102	Fall	2017
5	Snow	Biology	BIO-191	Spring	2017
6	NULL	NULL	HIS-120	Spring	2017

Next Lecture



University of Windsor

Next Lecture

- Hands-on to Database Programming in Java
 - MySQL (open source?)
 - Apache Derby (open source)



Part V

Training and resources





Tools



Tools

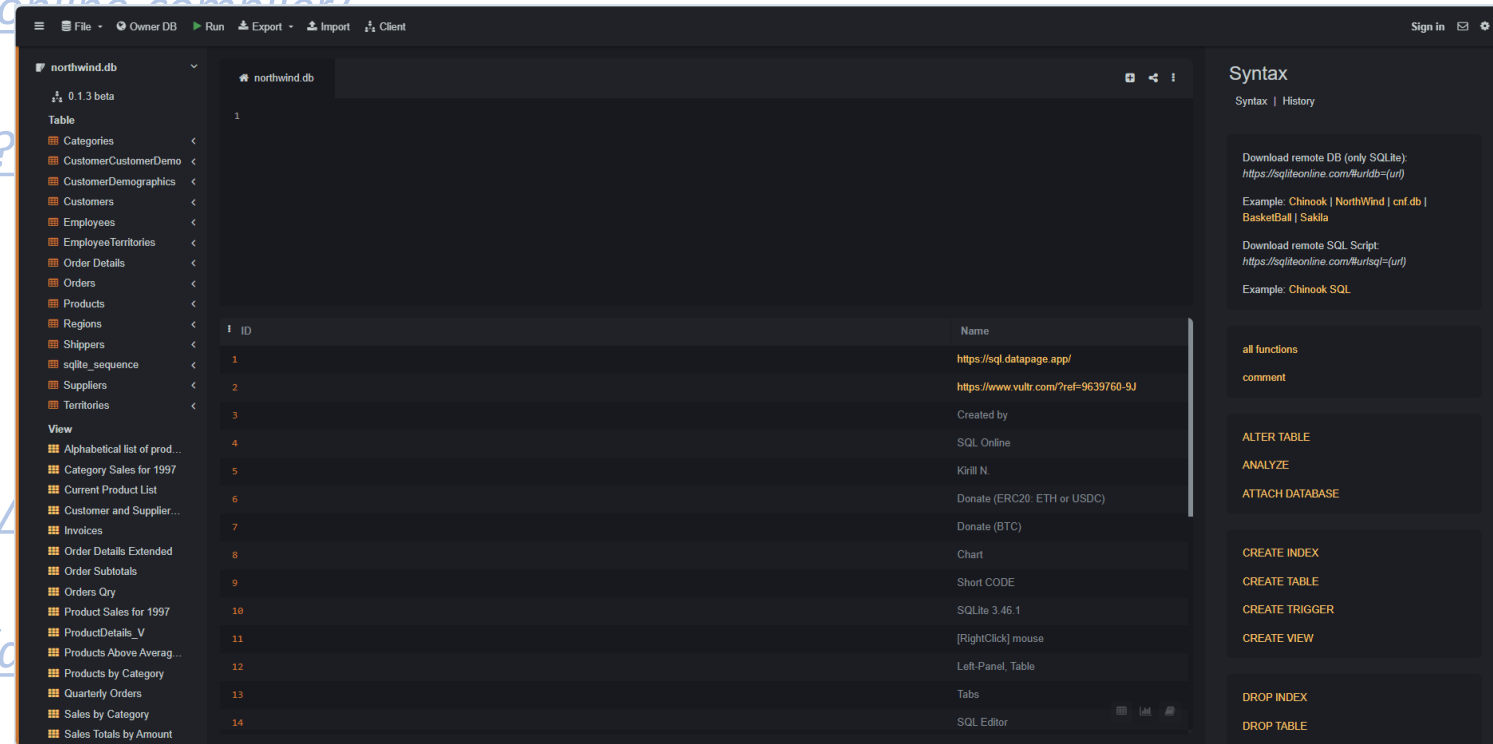
- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- https://www.w3schools.com/sql/sql_editor.asp
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- <https://livesql.oracle.com/apex/f?p=590:1000>
- Visual Studio Code
- Notepad++

- DBeaver: <https://dbeaver.io/>
- SqlDBM: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- Oracle: <https://www.oracle.com/database/sqldeveloper/technologies/sql-data-modeler/>



Tools

- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- https://www.w3schools.com/sql/sql_editor.asp
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- https://livesql.oracle.com/apex/f?_af=1
- *Visual Studio Code*
- *Notepad++*
- *DBeaver*: <https://dbeaver.io/>
- *SqlDBM*: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- *Oracle*: <https://www.oracle.com/>



○ Training resources (SQL)



Training Resources (SQL)

- Datacamp: <https://app.datacamp.com/learn/courses/introduction-to-sql>
- <https://www.essentialsql.com/>
- <https://www.w3schools.com/sql/>
- <https://www.khanacademy.org/computing/computer-programming/sql/sql-basics/v/welcome-to-sql>
- <https://www.learnsqlonline.org/>
- <https://sqlbolt.com/>
- https://sqlzoo.net/wiki/SQL_Tutorial
- <https://www.codecademy.com/resources/docs/sql>
- <https://www.freecodecamp.org/>
- ...



○ Training resources (JOINS)



Training Resources (JOINS)

- DataCamp:
<https://www.datacamp.com/tutorial/introduction-to-sql-joins>
- W3 Schools
- JOIN Visualizers:
 - Visual JOIN
 - SQL Joins Visualizer
 - SQL Query Visualizer: Parse Tree, Structure, and SQL
 - SQL JOIN Visualizer | Interactive SQL Tool - Doses of Data
 - Visual Representation of SQL joins
- MySQL Join Clause
- Join Operations in SQLite
- ...



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor